

PyTV Module Design Document

Module: ADD
Author: Bolin Li and Yifang Dai
Date: 2025/3/26
Version: 0.2.1

Revision History:

Version	Author	Date	Notes
0.1.1	Bolin Li	2024/11/03	Initial Update
0.1.2	Bolin Li	2024/11/04	Complete the interface part of the document, and major revise of the functionality part.
0.1.3	Bolin Li	2024/11/06	Redefine enable signal as a switch on input data signals, Small fixes of doc
0.2.1	Yifang Dai	2025/03/25	Overall adjustment and optimization (deleted enable signal)

Release Notes:

(optional) It is recommended to write brief update messages in the table above and provide detailed release notes here. Any important notations should be included.

Contents

1	Preliminaries	1
1.1	Purpose and scope	1
1.2	Notations	1
1.3	Typographic conventions	1
1.4	Dependencies	2
1.4.1	Software dependencies	2
1.4.2	Submodule dependencies	2
2	Functionality	3
2.1	Overview	3
2.2	Configurable parameters	3
2.2.1	Quantization sets	3
2.2.2	Quantization modes	3
2.2.3	Overflow modes	4
2.3	External interface	4
2.3.1	Clock and reset	4
2.3.2	Main Module Interface	4
2.4	Functional description	4
3	Implementation	5
3.1	Overall architecture	5
3.2	Data and control flow	5
3.3	External sequential behavior	5
3.3.1	Latency analysis	5
3.3.2	Throughput analysis	5

1 Preliminaries

1.1 Purpose and scope

This module will realize a general purpose **fixed-point addition** operation. It takes two fixed-point inputs and computes their sum in the specified fixed-point format.

1.2 Notations

All definitions, acronyms, and abbreviations used in the document should be listed here.

- LSB: Least significant bit.
- MSB: Most significant bit.
- TCPL: Two's complement format
- SMGN: Sign-Magnitude format.
- Functional description: A description of the module's combinational behavior.
- Functional model: A (usually QuBLAS) model of the module's behavior.

1.3 Typographic conventions

The following typographic conventions are used in this document:

Visual Cue	Meaning
(Bullet) •	List of important items.
monospace	Codes or commands typed on the keyboard.
MONOSPACE	Constants, parameters.
<i>italic</i>	Variable values, file and directory names.
<italic>	Placeholder text for users to insert values with.
[item]	Optional items.
bold red text	Important notes, warnings, or errors.
bold green text	Indicating success.
green text	Comments or remarks.
blue text	Revised text or keywords.
monospace/blue	URLs, links.
SMALL CAPS	Hardware submodule names.

1.4 Dependencies

Here lists all the dependencies of the module, including the software and submodule dependencies.

1.4.1 Software dependencies

This part lists all the software dependencies of the module, including the software name and version.

Software	Version
PyTV (verithon)	0.3
PyTU	0.1.0

1.4.2 Submodule dependencies

This part lists all the PyTV submodule dependencies of the module, including the submodule name and version.

Submodule	Version
DELAY	0.1.0
FxMATCH	0.2.1

2 Functionality

2.1 Overview

The ADD Module receives two input values with configurable precisions, then output their sum in a configurable precision fixed-point format. Now the module supports quantization modes and overflow modes listed in section 2.2.2 and 2.2.3, respectively. The module also supports optional **negedge reset**, which can be configured by IF_RST_N parameter, respectively.

2.2 Configurable parameters

Parameter	Range	Default	Description
QU_IN_1	$[\mathbb{N}^+, \mathbb{Z}, \mathbb{B}]$	[9, 3, True]	Summand data quantization set
QU_IN_2	$[\mathbb{N}^+, \mathbb{Z}, \mathbb{B}]$	[9, 3, True]	Addend data quantization set
QU_OUT	$[\mathbb{N}^+, \mathbb{Z}, \mathbb{B}]$	[9, 3, 1]	Output quantization set
QU_MODE	All available modes in 2.2.2	QuMode.TRN.TCPL	Output FxMatch Quantize Mode
OF_MODE	All available modes in 2.2.3	OfMode.WRP.TCPL	Output FxMatch Overflow Mode
N_PIPELINES	$\mathbb{N}$	0	Number of pipeline stages
IF_RST_N	$\mathbb{B}$	False	Sign for including negedge reset
	$[\mathbb{B}, \dots]$	[False, ...]	Requires N_PIPELINES > 0

2.2.1 Quantization sets

Quantization set describes the quantization format of value, which is originated from a PyTU class QuType. Here is a list of QuType's members:

Member	Range	Description
DWT	$\mathbb{N}^+$	Data Width
FRAC	$\mathbb{Z}$	Decimal Point Offset, in bits
IF_SIGNED	$\mathbb{B}$	Signed or unsigned input format

2.2.2 Quantization modes

Quantization mode refers to the method applied when dealing with precision loss (at LSB). This module supported the following modes:

Mode	Description
QuMode.TRN.TCPL	Truncate all extra LSB in TCPL format
QuMode.TRN.SMGN	Truncate all extra LSB in Sign-Magnitude (SMGN) format, then convert back to TCPL format
QuMode.RND.POS_INF	Round to nearest quantization level, when the value is equidistant between two levels, round to the larger value
QuMode.RND.NEG_INF	Round to nearest quantization level, when the value is equidistant between two levels, round to the smaller value
QuMode.RND.ZERO	Round to nearest quantization level, when the value is equidistant between two levels, round to the value closer to zero
QuMode.RND.INF	Round to nearest quantization level, when the value is equidistant between two levels, round to the value further from zero

QuMode.RND.CONV

Round to nearest quantization level, when the value is equidistant between two levels, round to the even value (LSB = 0)

2.2.3 Overflow modes

Overflow mode refers to the method applied when dealing with overflow (at MSB). This module supported the following modes:

Mode	Description
OfMode.WRP.TCPL	Wrap around in TCPL format
OfMode.SAT.TCPL	Saturate to the maximum value in TCPL format
OfMode.SAT.SMGN	Saturate to the maximum value in SMGN format, then convert back to TCPL
OfMode.SAT.ZERO	Saturate to zero

2.3 External interface

2.3.1 Clock and reset

Synchronous reset and asynchronous release is the default reset behavior. If the module has a different reset behavior, it should be described here.

Signal	Width	Description
i_clk	1	Clock signal (Requires N_PIPELINES >0)
i_rst_n	1	Reset signal (Requires N_PIPELINES >0)

2.3.2 Main Module Interface

Signal	Width	Description
i_data_1	QU_IN_1.DWT	Input signal for summand data
i_data_2	QU_IN_2.DWT	Input signal for addend data
o_result	QU_OUT.DWT	Output signal for result

2.4 Functional description

Here depicts a functional description of the module.

Algorithm 1: Module function

```

Input: i_data_1
Input: i_data_2
Output: o_result

1 data_1 = i_data_1;
2 data_2 = i_data_2;
  // Adder
3 result_unfixed = data_1 + data_2;
  // Quantize and handle overflow to get the specified-format final result
4 o_result = result_unfixed.FxMatch(o_result.DWT, o_result.FRAC, o_result.SIGNED);

```

3 Implementation

3.1 Overall architecture

The ADD Module relies on the FxMATCH module to perform the fixed-point addition operation. The FxMATCH module takes two input values with configurable precisions, then output their sum in a configurable precision fixed-point format. The ADD Module then applies the specified quantization and overflow modes to the output value.

When N_PIPELINES is 0, the module is a combinational module and the output value is directly generated in the combinational logic. When N_PIPELINES is greater than 0, the module is a pipelined module with N_PIPELINES pipeline stages, and the output value is generated in a pipeline stage.

3.2 Data and control flow

The module is either a combinational module (when N_PIPELINES is 0) or a fully pipelined module (when N_PIPELINES > 0).

3.3 External sequential behavior

3.3.1 Latency analysis

Latency are N_PIPELINES clock cycles for output the result.

3.3.2 Throughput analysis

This module can handle one frame per clock cycle.